

整理文档：vbench 2026-01-11

VBench 核心评测维度技术文档

本文档详细介绍VBench框架中5个核心评测维度的实现原理、评测流程和应用场景。

目录

- Temporal Flickering - 时序闪烁
- Motion Smoothness - 运动平滑度
- Subject Consistency - 主体一致性
- Background Consistency - 背景一致性
- Overall Consistency - 整体一致性

1. Temporal Flickering - 时序闪烁

1.1 概述

定义: 评估视频在时间维度上的纹理闪烁和抖动程度。

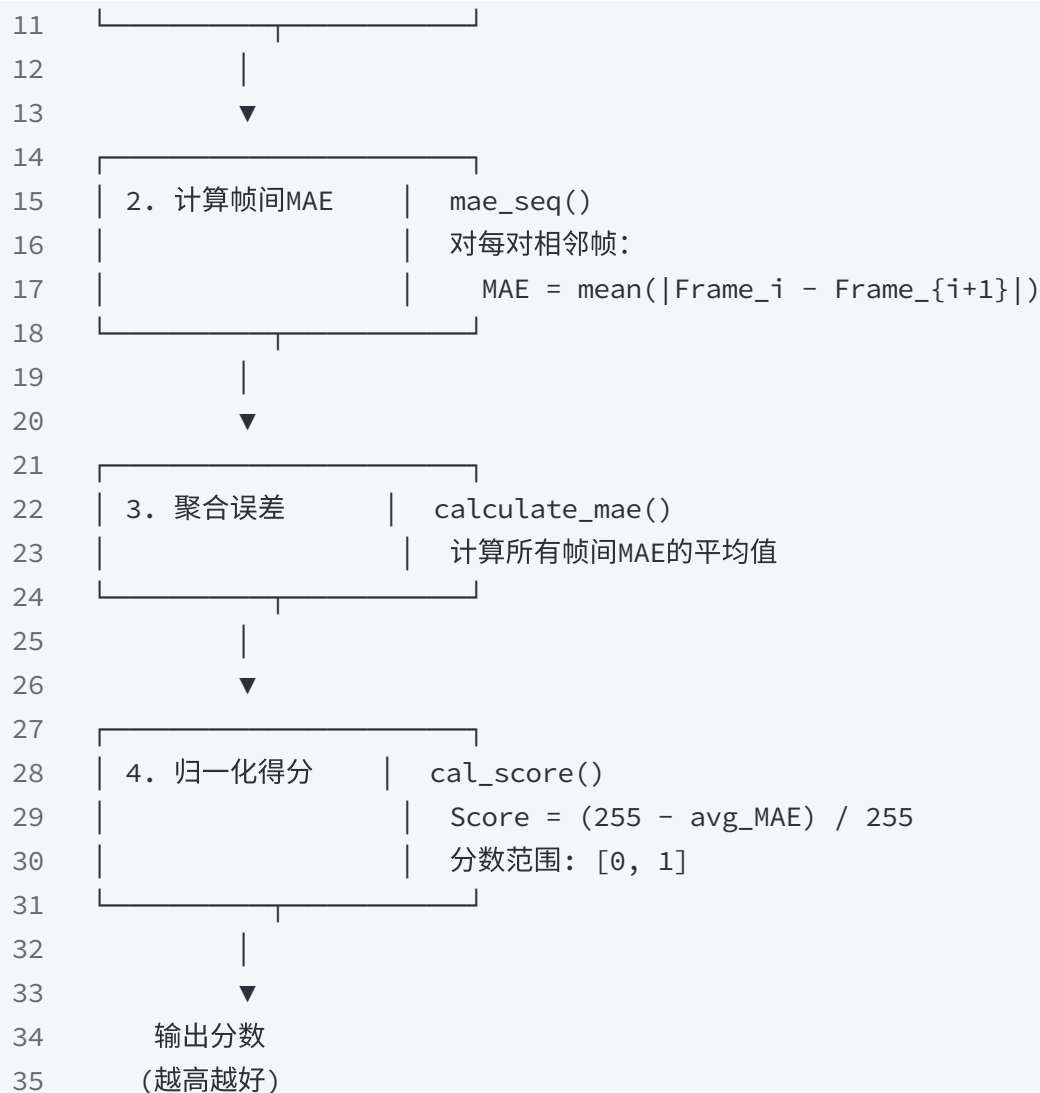
代码路径: vbench/temporal_flickering.py

核心思想: 通过计算相邻帧之间的平均绝对误差(MAE)来量化闪烁程度。假设视频内容是静态的，任何帧间差异都被视为闪烁。

1.2 评测流程图

代码块





1.3 核心算法

1.3.1 帧提取

代码块

```
1 def get_frames(video_path):
2     """提取视频的所有帧"""
3     frames = []
4     video = cv2.VideoCapture(video_path)
5     while video.isOpened():
6         success, frame = video.read()
7         if success:
8             frames.append(frame)
9         else:
10            break
11    video.release()
12    return frames
```

1.3.2 MAE计算

代码块

```
1 def calculate_mae(img1, img2):
2     """计算两帧之间的平均绝对误差""" # MAE = mean(|img1 - img2|)
3     return np.mean(cv2.absdiff(
4         np.array(img1, dtype=np.float32),
5         np.array(img2, dtype=np.float32)
6     ))
```

数学公式:

$$MAE(Frame_i, Frame_{i+1}) = \frac{1}{H \times W \times C} \sum_{h,w,c} |Frame_i(h,w,c) - Frame_{i+1}(h,w,c)|$$
$$MAE(Frame_i, Frame_{i+1}) = \frac{1}{H \times W \times C} \sum_{h,w,c} |Frame_i(h,w,c) - Frame_{i+1}(h,w,c)|$$

其中:

- H, W, C 分别为帧的高度、宽度和通道数
- h, w, c 为像素位置和通道索引

1.3.3 序列MAE计算

代码块

```
1 def mae_seq(frames):
2     """计算所有相邻帧对的MAE"""
3     ssds = []
4     for i in range(len(frames)-1):
5         ssds.append(calculate_mae(frames[i], frames[i+1]))
6     return np.array(ssds)
```

1.3.4 最终得分计算

代码块

```
1 def cal_score(video_path):
2     """计算视频的闪烁得分"""
3     frames = get_frames(video_path)
4     score_seq = mae_seq(frames)
5     # 归一化到[0,1], 分数越高越好
6     return (255.0 - np.mean(score_seq).item()) / 255.0
```

最终得分公式:

$$Score_{TF} = 255 - MAE^{-1} \cdot 255$$
$$Score_{TF} = 255 - MAE$$

其中 MAE^{-1} 是所有相邻帧MAE的平均值。

1.4 技术特点

优点:

-  算法简单，计算速度快
-  不依赖深度学习模型，资源消耗低
-  适合批量快速评测

局限性:

-  假设视频内容静态，不适用于运动场景
-  对相机运动非常敏感（会误判为闪烁）
-  无法区分真实内容变化和闪烁

1.5 应用场景

1.6 实际应用示例

代码块

```
1  from vbench import VBench
2
3  # 初始化VBench
4  vbench = VBench('cuda', 'vbench/VBench_full_info.json', 'results')
5
6  # 评测静态场景视频
7  vbench.evaluate(
8      videos_path='static_videos/',
9      name='temporal_flickering_test',
10     dimension_list=['temporal_flickering'],
11     mode='custom_input'
12 )
```

1.7 评分解读

2. Motion Smoothness - 运动平滑度

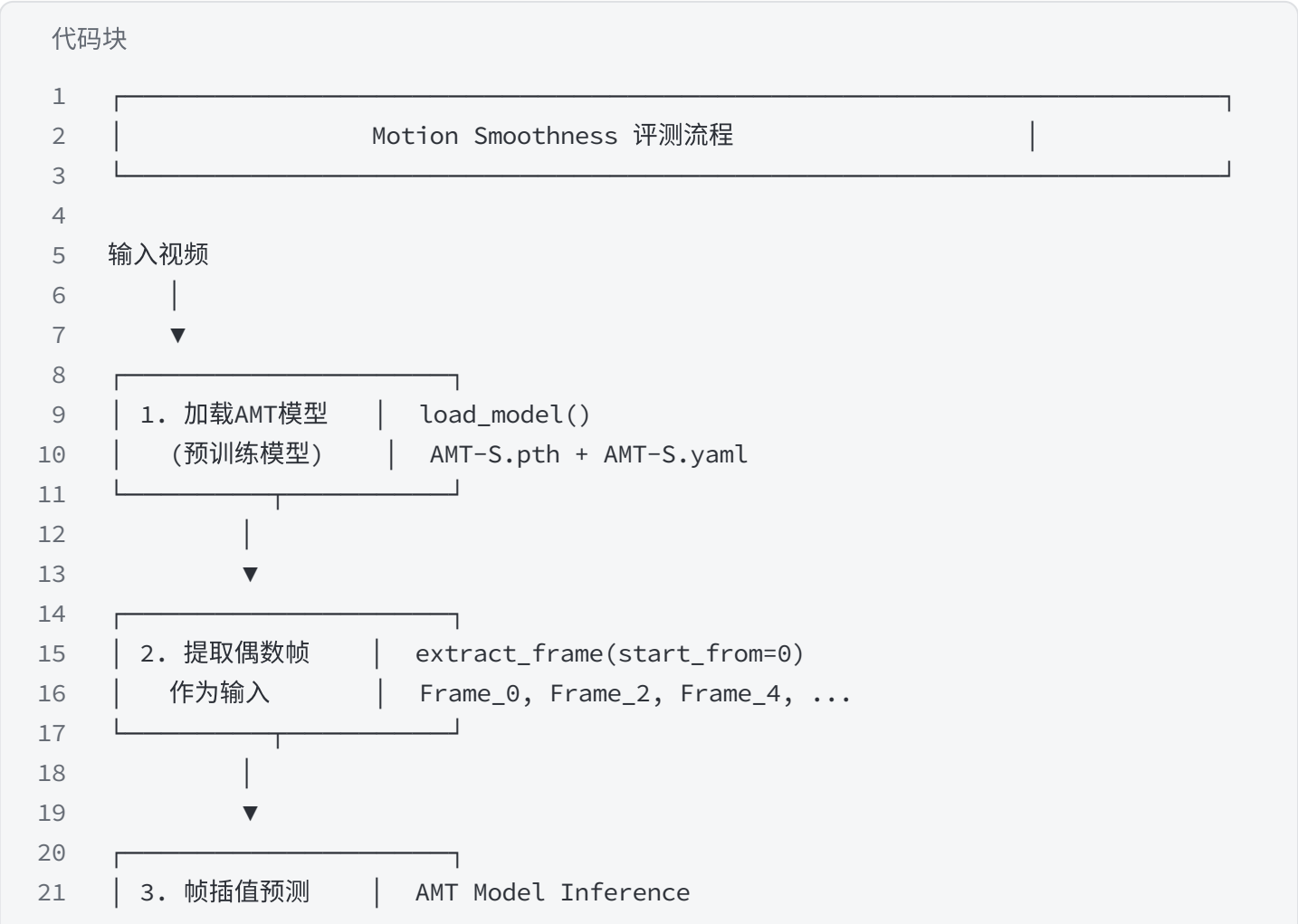
2.1 概述

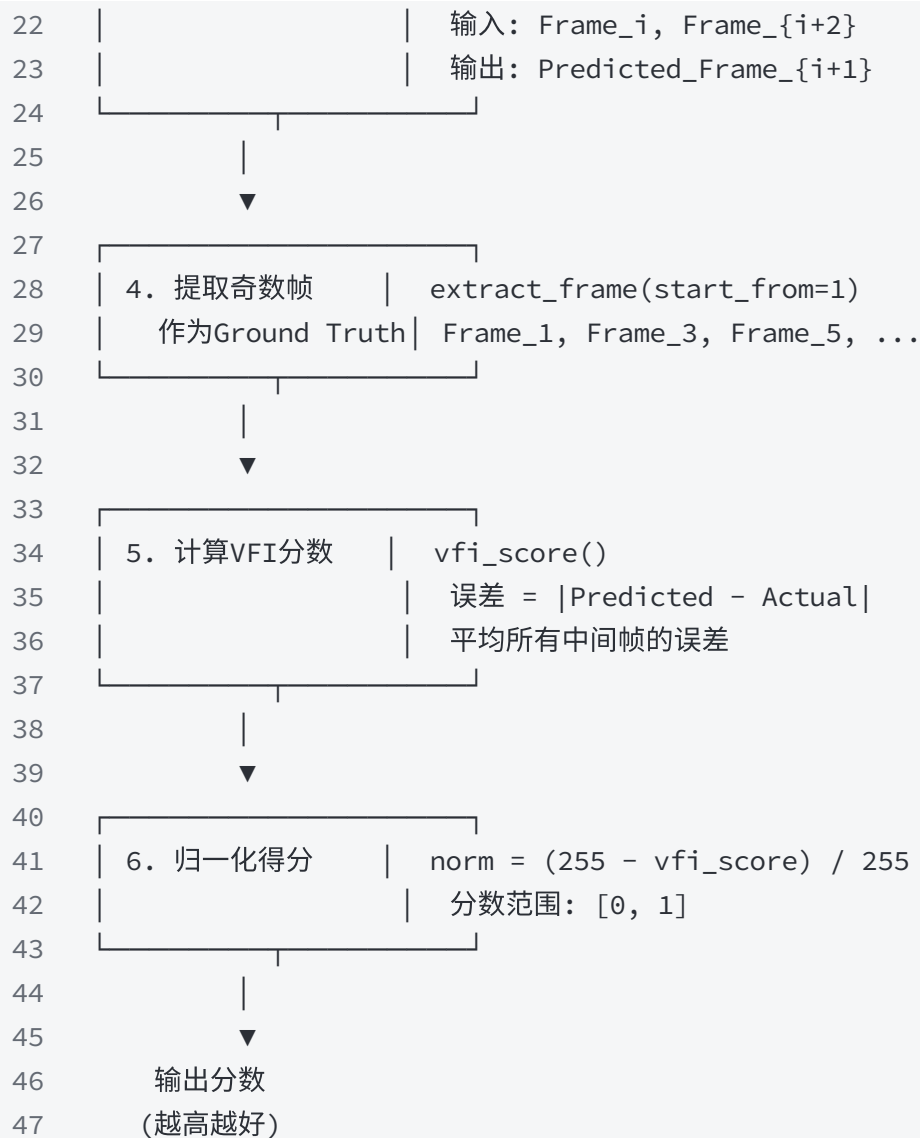
定义: 评估视频中运动的平滑程度和连续性，主要用于检测运动抖动、卡顿和不自然的加速度变化。

代码路径: `vbench/motion_smoothness.py`

核心思想: 使用AMT (Any-resolution, Motion-aware, Temporal-interpolation)深度学习模型进行帧插值，通过比较插值预测的中间帧与实际中间帧的差异来评估运动平滑度。

2.2 评测流程图





2.3 核心算法

2.3.1 AMT模型初始化

代码块

```
1 class MotionSmoothness:
2     def __init__(self, config, ckpt, device):
3         self.device = device
4         self.config = config # AMT-S.yaml
5         self.ckpt = ckpt # amt-s.pth
6         self.niters = 1
7         self.load_model()
```

2.3.2 帧插值过程

代码块

```

1  def motion_score(self, video_path):
2      # 1. 提取偶数帧作为输入
3      frames = self.fp.get_frames(video_path)
4      frame_list = self.fp.extract_frame(frames, start_from=0)
5      inputs = [img2tensor(frame).to(self.device) for frame in frame_list]
6
7      # 2. 使用AMT模型进行插值
8      outputs = [inputs[0]]
9      for in_0, in_1 in zip(inputs[:-1], inputs[1:]):
10         with torch.no_grad():
11             # 预测中间帧
12             imgt_pred = self.model(
13                 in_0, in_1,
14                 self.embt, # t=0.5, 预测正中间的帧
15                 scale_factor=scale,
16                 eval=True
17             )['imgt_pred']
18             outputs += [imgt_pred.cpu(), in_1.cpu()]
19
20     # 3. 计算VFI分数
21     outputs = [tensor2img(out) for out in outputs]
22     vfi_score = self.vfi_score(frames, outputs)
23     norm = (255.0 - vfi_score) / 255.0 return norm

```

2.3.3 VFI分数计算

代码块

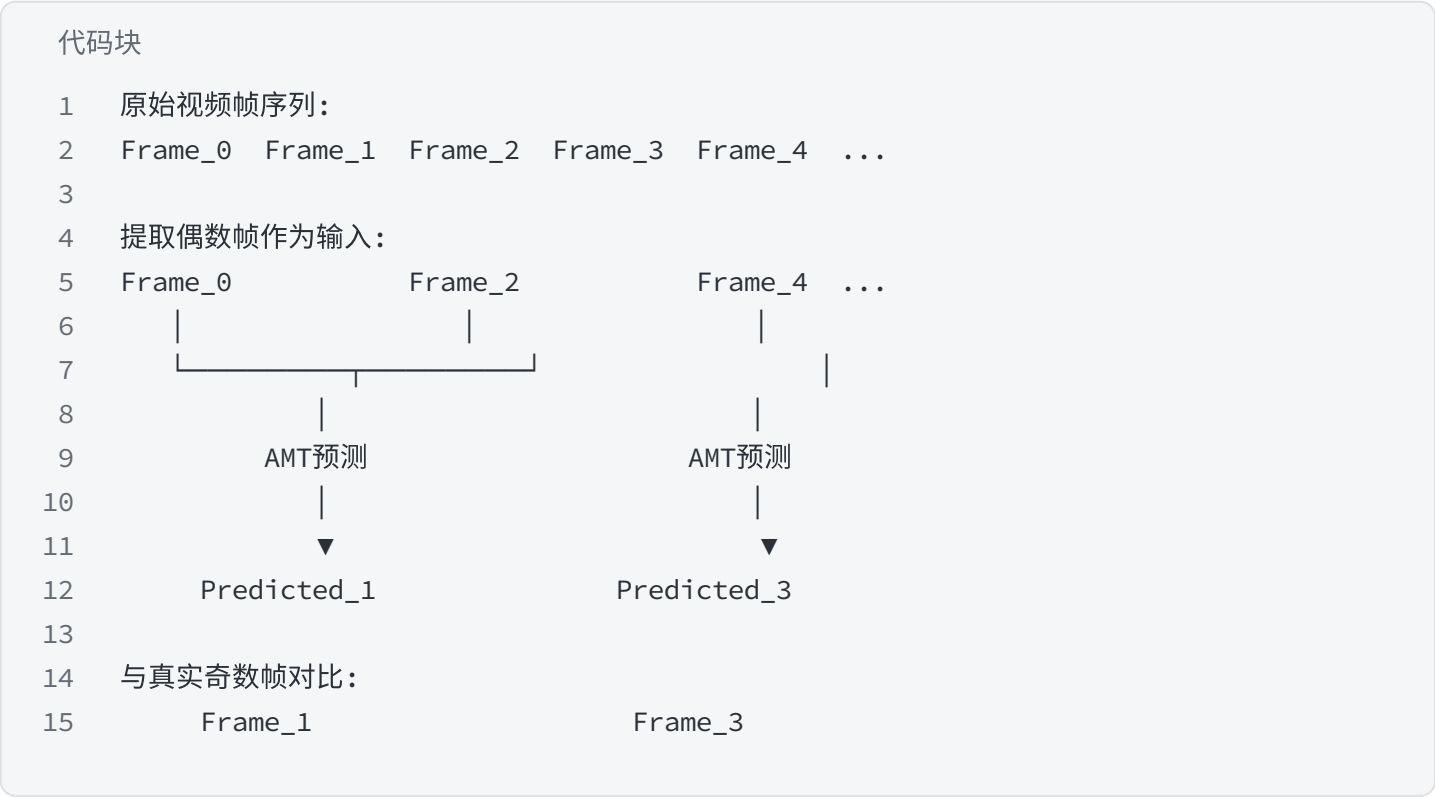
```

1  def vfi_score(self, ori_frames, interpolate_frames):
2      """计算Video Frame Interpolation分数"""# 提取奇数帧 (Ground Truth)
3      ori = self.fp.extract_frame(ori_frames, start_from=1)
4      # 提取预测的中间帧
5      interpolate = self.fp.extract_frame(interpolate_frames, start_from=1)
6
7      scores = []
8      for i in range(len(interpolate)):
9          # 计算预测帧与真实帧的差异
10         scores.append(self.get_diff(ori[i], interpolate[i]))
11
12     return np.mean(np.array(scores))
13
14 def get_diff(self, img1, img2):
15     """计算两帧的平均绝对差异"""
16     img = cv2.absdiff(img1, img2)
17     return np.mean(img)

```

2.4 技术原理详解

2.4.1 帧插值原理



2.4.2 平滑度评估原理

核心假设: 如果视频运动平滑，则通过相邻帧可以准确预测中间帧。

- **运动平滑** → 预测准确 → 误差小 → 分数高
- **运动抖动** → 预测失败 → 误差大 → 分数低

数学公式:

$$VFerror = \frac{1}{N} \sum_{i=1}^N |Frame_{predicted_i} - Frame_{actual_i}|$$
$$Score_{MS} = \frac{255 - VFerror}{255}$$

2.5 技术特点

优点:

- ☒ 专门评估运动平滑性
- ☒ 基于深度学习，感知能力强
- ☒ 能检测微小的运动不连续性
- ☒ 直接反映防抖性能

局限性:

- ⚠ 需要预训练的AMT模型 (~200MB)

- ⚠ GPU显存需求较高 (4-6GB)
- ⚠ 计算速度较慢 (~0.3-0.5 FPS)
- ⚠ 对快速运动场景，插值可能不准确

2.6 应用场景

2.7 实际应用示例

代码块

```
1  from vbench import VBench
2
3  vbench = VBench('cuda', 'vbench/VBench_full_info.json', 'results')
4
5  # 评测防抖性能
6  vbench.evaluate(
7      videos_path='handheld_videos/',
8      name='motion_smoothness_test',
9      dimension_list=['motion_smoothness'],
10     mode='custom_input'
11 )
```

2.8 评分解读

3. Subject Consistency - 主体一致性

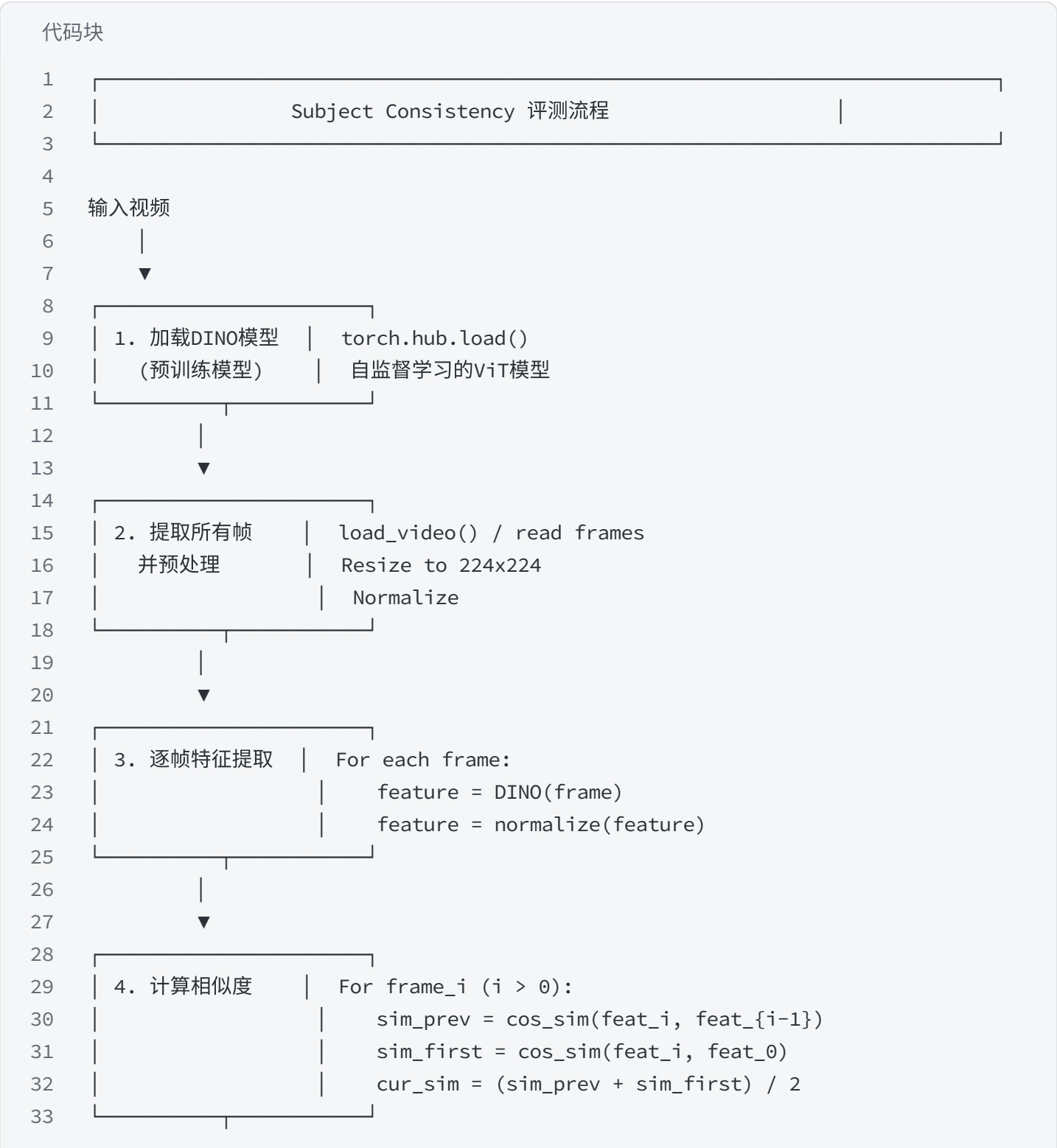
3.1 概述

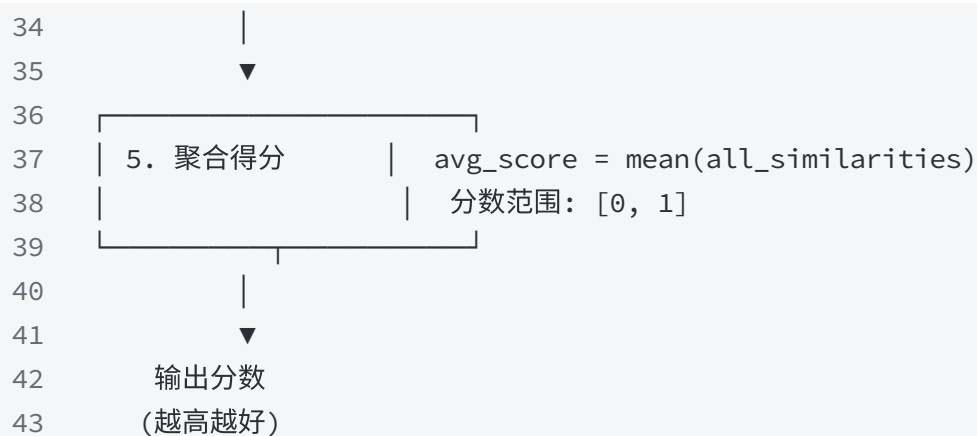
定义: 评估视频中主体对象在时间维度上的一致性，检测主体的外观、特征是否保持稳定。

代码路径: `vbench/subject_consistency.py`

核心思想: 使用DINO (Self-Distillation with NO labels)模型提取主体特征，通过计算帧间余弦相似度来评估主体的时序一致性。

3.2 评测流程图





3.3 核心算法

3.3.1 DINO特征提取

代码块

```

1  def subject_consistency(model, video_list, device, read_frame):
2      sim = 0.0
3      cnt = 0
4      image_transform = dino_transform(224)
5
6      for video_path in video_list:
7          video_sim = 0.0# 加载视频帧
8          images = load_video(video_path)
9          images = image_transform(images)
10
11         # 逐帧提取特征for i in range(len(images)):
12             with torch.no_grad():
13                 image = images[i].unsqueeze(0).to(device)
14                 # DINO特征提取
15                 image_features = model(image)
16                 # L2归一化
17                 image_features = F.normalize(image_features, dim=-1, p=2)
18
19             # 后续处理...

```

3.3.2 相似度计算

代码块

```
1 for i in range(len(images)):
2     # 提取当前帧特征
3     image_features = model(image)
4     image_features = F.normalize(image_features, dim=-1, p=2)
```

```

5
6     if i == 0:
7         # 保存第一帧特征作为参考
8         first_image_features = image_features
9     else:
10        # 与前一帧的相似度
11        sim_pre = max(0.0, F.cosine_similarity(
12            former_image_features,
13            image_features
14        ).item())
15
16        # 与第一帧的相似度
17        sim_fir = max(0.0, F.cosine_similarity(
18            first_image_features,
19            image_features
20        ).item())
21
22        # 综合相似度
23        cur_sim = (sim_pre + sim_fir) / 2
24        video_sim += cur_sim
25        cnt += 1
26
27        former_image_features = image_features

```

3.4 技术原理详解

3.4.1 DINO模型特点

DINO (Self-Distillation with NO labels):

- 基于Vision Transformer (ViT)架构
- 自监督学习，无需标注数据
- 擅长提取语义特征
- 对主体对象的特征捕捉能力强

模型加载:

代码块

```

1 dino_model = torch.hub.load(
2     'facebookresearch/dino:main',
3     'dino_vits16'
4 )

```

3.4.2 相似度计算策略

双重比较策略:

1. 与前一帧比较 (sim_pre):

- 评估短期一致性
- 检测帧间突变

2. 与第一帧比较 (sim_fir):

- 评估长期一致性
- 防止累积漂移

最终相似度:

$$Sim_i = Sim_{prev}(i, i-1) + Sim_{first}(i, 0) \quad 2Sim_i = 2Sim_{prev}(i, i-1) + Sim_{first}(i, 0)$$

余弦相似度公式:

$$\cos(A,B) = \frac{A \cdot B}{|A| \cdot |B|} \quad \cos(A,B) = \frac{A \cdot B}{|A| \cdot |B|}$$

3.5 技术特点

优点:

- ☒ 基于自监督学习的DINO，特征提取能力强
- ☒ 双重比较策略，兼顾短期和长期一致性
- ☒ 对主体外观变化敏感
- ☒ 不需要文本prompt

局限性:

- ⚠ 需要DINO预训练模型
- ⚠ 对相机运动敏感（可能影响主体特征）
- ⚠ 如果视频中主体较小，可能检测不准确
- ⚠ GPU显存需求中等（~2-3GB）

3.6 应用场景

3.7 实际应用示例

代码块

```
1  from vbench import VBench
2
3  vbench = VBench('cuda', 'vbench/VBench_full_info.json', 'results')
4
5  # 评测主体一致性
6  vbench.evaluate(
7      videos_path='portrait_videos/',
8      name='subject_consistency_test',
9      dimension_list=['subject_consistency'],
10     mode='custom_input'
11 )
```

3.8 评分解读

4. Background Consistency - 背景一致性

4.1 概述

定义: 评估视频中背景在时间维度上的一致性，检测背景的稳定性和连续性。

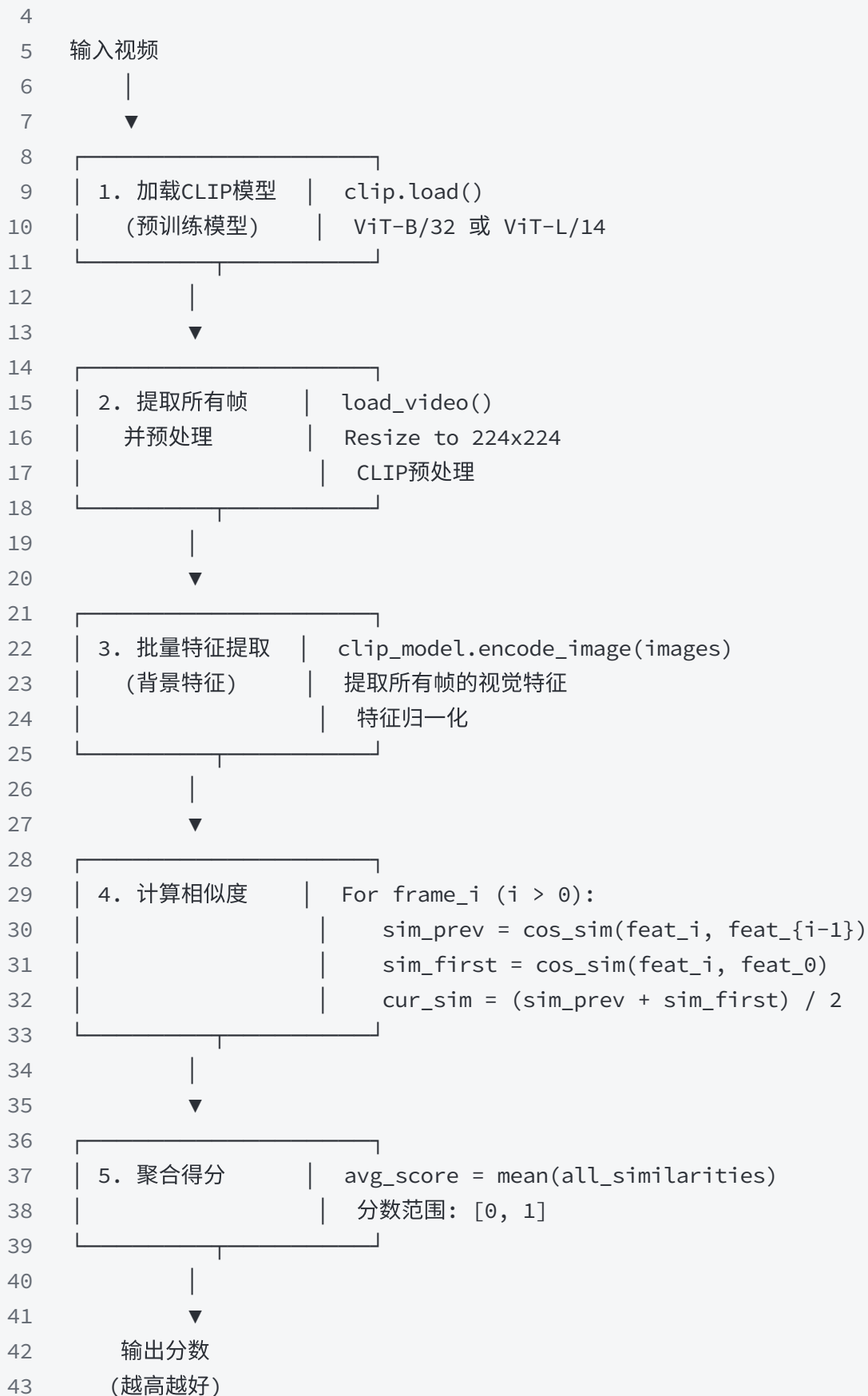
代码路径: vbench/background_consistency.py

核心思想: 使用CLIP (Contrastive Language-Image Pre-training)模型提取背景特征，通过计算帧间余弦相似度来评估背景的时序一致性。

4.2 评测流程图

代码块

```
1  [ Background Consistency 评测流程 ]
2
3
```



4.3 核心算法

4.3.1 CLIP特征提取

```

1  def background_consistency(clip_model, preprocess, video_list, device,
    read_frame):
2      sim = 0.0
3      cnt = 0
4      image_transform = clip_transform(224)
5
6      for video_path in video_list:
7          video_sim = 0.0
8          cnt_per_video = 0 # 加载并预处理视频帧
9          images = load_video(video_path)
10         images = image_transform(images)
11         images = images.to(device)
12
13         # 批量提取所有帧的特征
14         image_features = clip_model.encode_image(images)
15         image_features = F.normalize(image_features, dim=-1, p=2)
16
17         # 后续处理...

```

4.3.2 背景一致性计算

代码块

```

1  # 逐帧计算相似度 for i in range(len(image_features)):
2      image_feature = image_features[i].unsqueeze(0)
3
4      if i == 0:
5          # 保存第一帧特征
6          first_image_feature = image_feature
7      else:
8          # 与前一帧的相似度
9          sim_pre = max(0.0, F.cosine_similarity(
10              former_image_feature,
11              image_feature
12          ).item())
13
14          # 与第一帧的相似度
15          sim_fir = max(0.0, F.cosine_similarity(
16              first_image_feature,
17              image_feature
18          ).item())
19
20          # 综合相似度
21          cur_sim = (sim_pre + sim_fir) / 2
22          video_sim += cur_sim
23          cnt += 1

```



```
24         cnt_per_video += 1
25
26     former_image_feature = image_feature
27
28     # 计算平均相似度
29     sim_per_image = video_sim / (len(image_features) - 1)
```

4.4 技术原理详解

4.4.1 CLIP模型特点

CLIP (Contrastive Language-Image Pre-training):

- 通过对比学习训练的多模态模型
- 可以同时理解图像和文本
- 擅长提取图像的全局语义特征
- 对背景场景的特征捕捉能力强

模型加载:

代码块

```
1 clip_model, preprocess = clip.load('ViT-B/32', device=device)
```

可用模型:

- ViT-B/32: 快速，适合实时应用
- ViT-B/16: 平衡速度和精度
- ViT-L/14: 高精度，较慢

4.4.2 批量处理优势

与Subject Consistency不同，Background Consistency采用批量处理:

代码块

```
1 # 一次性编码所有帧
2 image_features = clip_model.encode_image(images)
3 # 再逐个计算相似度
```

优点:

- 更高的GPU利用率
- 更快的处理速度

- 适合长视频评测

4.5 技术特点

优点:

- ☒ CLIP模型对场景理解能力强
- ☒ 批量处理，速度较快
- ☒ 双重比较策略，兼顾短期和长期一致性
- ☒ 对背景变化敏感

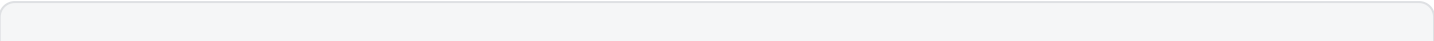
局限性:

- ⚠ 需要CLIP预训练模型 (~500MB)
- ⚠ GPU显存需求较高 (~4GB)
- ⚠ 对前景主体的变化也会有一定反应
- ⚠ 无法区分背景的真实变化和不一致性

4.6 应用场景

4.7 与Subject Consistency的对比

4.8 实际应用示例



```
1  # 从 vbench 导入 VBench
2
3  vbench = VBench('cuda', 'vbench/VBench_full_info.json', 'results')
4
5  # 评测背景一致性
6  vbench.evaluate(
7      videos_path='landscape_videos/',
8      name='background_consistency_test',
9      dimension_list=['background_consistency'],
10     mode='custom_input'
11 )
```

4.9 评分解读

5. Overall Consistency - 整体一致性

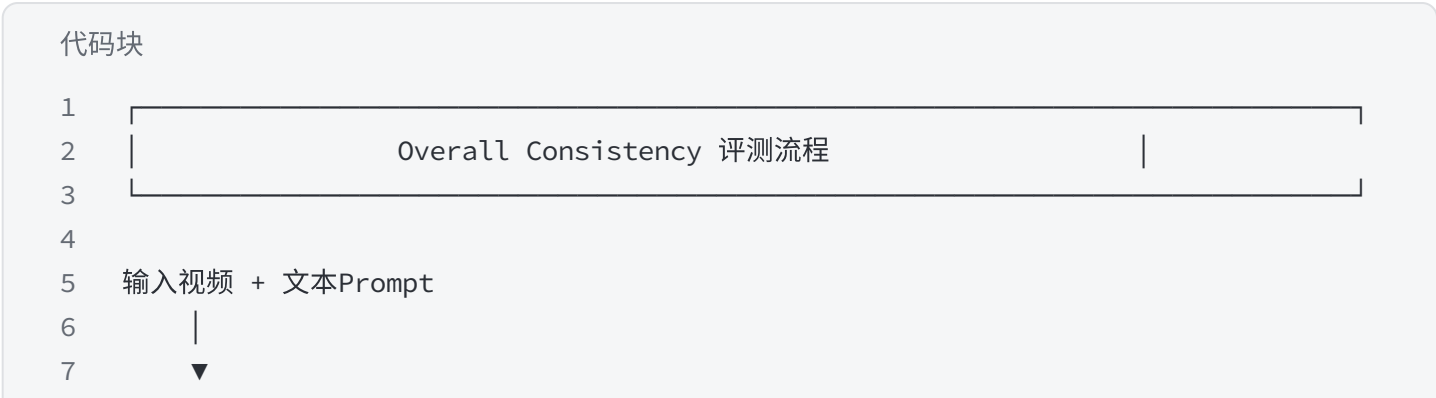
5.1 概述

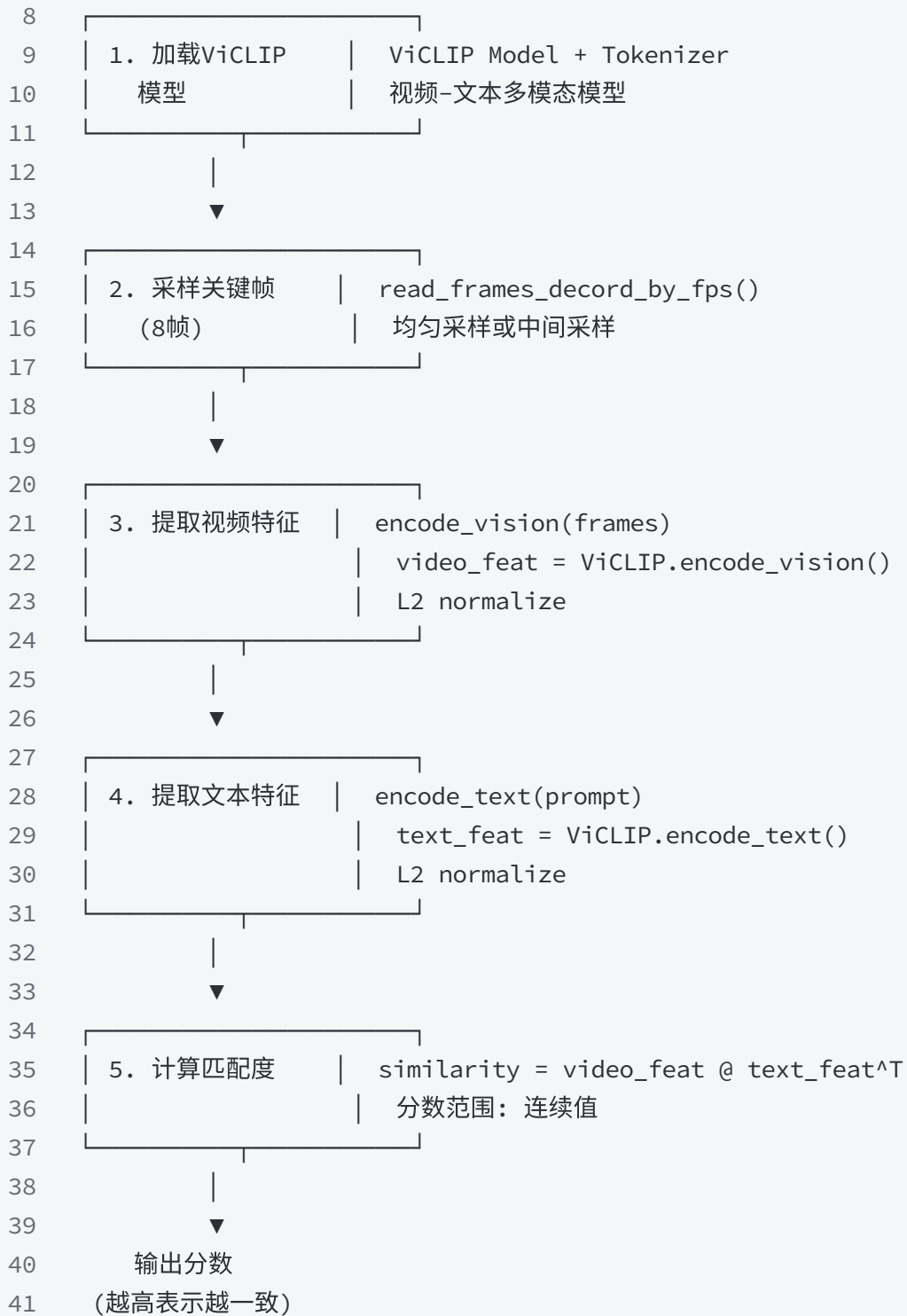
定义: 评估视频内容与文本prompt的整体一致性，以及视频在时间维度上的全局连贯性。

代码路径: vbench/overall_consistency.py

核心思想: 使用ViCLIP (Video-CLIP)多模态模型，计算视频特征与文本prompt特征之间的相似度，评估视频内容与描述的匹配程度。

5.2 评测流程图





5.3 核心算法

5.3.1 ViCLIP模型初始化

代码块

```
1 def compute_overall_consistency(json_dir, device, submodules_list, **kwargs):
2     # 加载ViCLIP tokenizer
3     tokenizer = SimpleTokenizer(
4         os.path.join(CACHE_DIR, "ViCLIP/bpe_simple_vocab_16e6.txt.gz")
5     )
```

```

6
7     # 加载ViCLIP模型
8     viclip = ViCLIP(tokenizer=tokenizer, **submodules_list).to(device)
9
10    # 加载视频和prompt信息
11    _, video_dict = load_dimension_info(
12        json_dir,
13        dimension='overall_consistency',
14        lang='en'
15    )

```

5.3.2 视频特征提取

代码块

```

1  def get_vid_features(model, input_frames):
2      """提取视频特征"""with torch.no_grad():
3          # 编码视频帧
4          clip_feat = model.encode_vision(input_frames, test=True).float()
5          # L2归一化
6          clip_feat /= clip_feat.norm(dim=-1, keepdim=True)
7      return clip_feat

```

5.3.3 文本特征提取

代码块

```

1  def get_text_features(model, input_text, tokenizer, text_feature_dict={}):
2      """提取文本特征（带缓存）"""if input_text in text_feature_dict:
3          return text_feature_dict[input_text]
4
5      text_template = f"{input_text}"with torch.no_grad():
6          # 编码文本
7          text_features = model.encode_text(text_template).float()
8          # L2归一化
9          text_features /= text_features.norm(dim=-1, keepdim=True)
10         # 缓存结果
11         text_feature_dict[input_text] = text_features
12     return text_features

```

5.3.4 整体一致性计算

代码块

```

1  def overall_consistency(clip_model, video_dict, tokenizer, device,

```

```

sample="middle"):
2     sim = []
3     video_results = []
4     image_transform = clip_transform(224)
5
6     for info in video_dict:
7         query = info['prompt'] # 文本描述
8         video_list = info['video_list']
9
10        for video_path in video_list:
11            with torch.no_grad():
12                # 1. 采样8帧
13                images = read_frames_decord_by_fps(
14                    video_path,
15                    num_frames=8,
16                    sample=sample
17                )
18                images = image_transform(images)
19                images = images.to(device)
20
21                # 2. 提取视频特征
22                clip_feat = get_vid_features(clip_model, images.unsqueeze(0))
23
24                # 3. 提取文本特征
25                text_feat = get_text_features(clip_model, query, tokenizer)
26
27                # 4. 计算视频-文本匹配度
28                logit_per_text = clip_feat @ text_feat.T
29                score_per_video = float(logit_per_text[0][0].cpu())
30
31                sim.append(score_per_video)
32                video_results.append({
33                    'video_path': video_path,
34                    'video_results': score_per_video
35                })
36
37    avg_score = np.mean(sim)
38    return avg_score, video_results

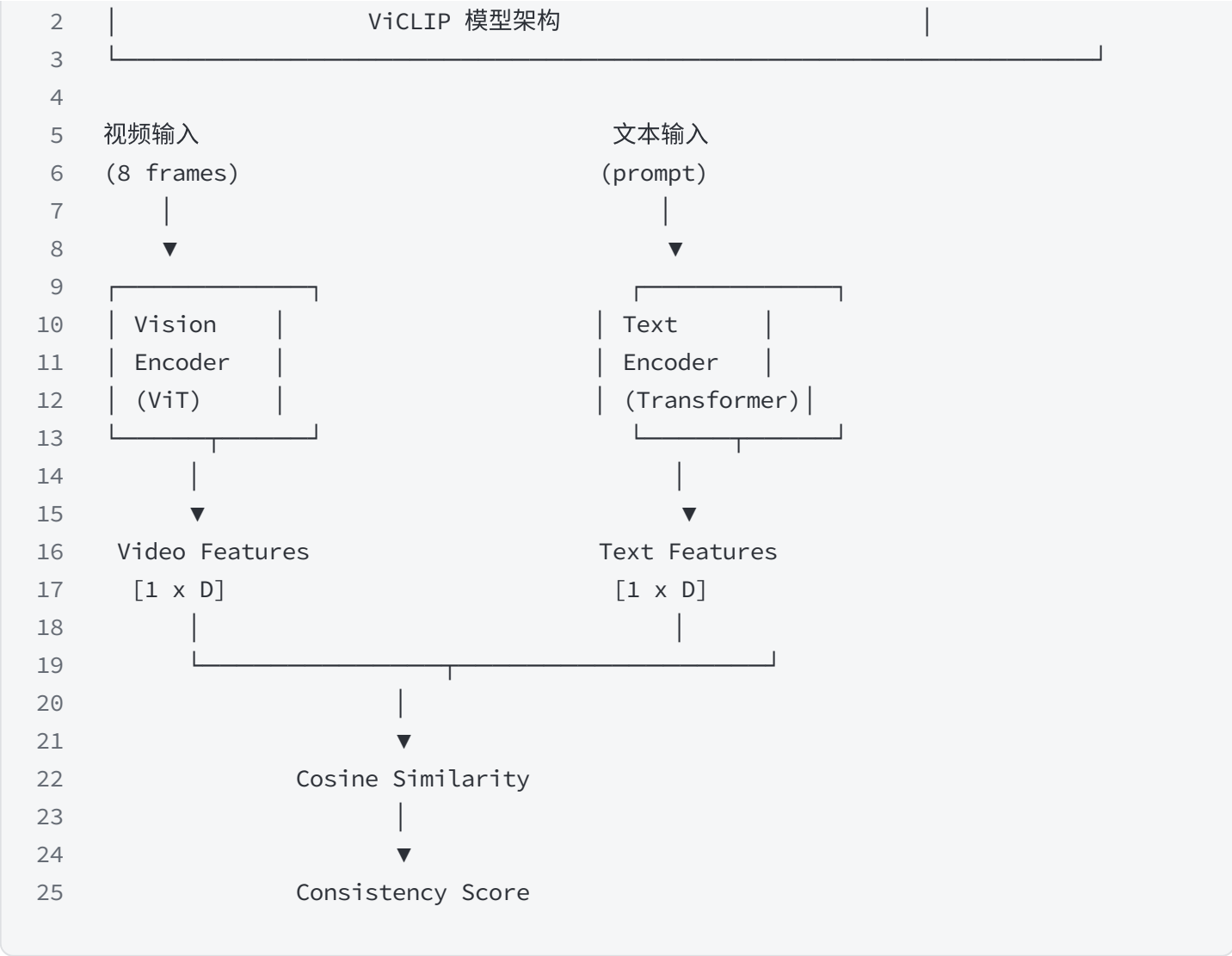
```

5.4 技术原理详解

5.4.1 ViCLIP模型架构

代码块

1



5.4.2 帧采样策略

两种采样模式:

1. 均匀采样 (uniform):

代码块

```
1 # 从整个视频均匀采样8帧
2 Frame indices: [0, N/7, 2N/7, 3N/7, 4N/7, 5N/7, 6N/7, N-1]
```

2. 中间采样 (middle):

代码块

```
1 # 从视频中间部分采样8帧
2 Frame indices: [N/4, N/4+Δ, ..., 3N/4]
```

5.4.3 相似度计算公式

视频-文本匹配度:

$$Score = V \cdot T + ||V|| \cdot ||T||$$
$$Score = ||V|| \cdot ||T|| + V \cdot T$$

其中:

- V 是视频特征向量
- T 是文本特征向量
- $\cdot$ 表示点积
- $|| \cdot ||$ 表示L2范数

5.5 技术特点

优点:

- ☒ 多模态模型，同时理解视频和文本
- ☒ 评估全局一致性，不局限于局部细节
- ☒ 只需采样8帧，计算效率高
- ☒ 文本特征可缓存，加速批量评测

局限性:

- ⚠ 需要文本prompt作为输入
- ⚠ 只采样8帧，可能错过细节问题
- ⚠ 对文本描述的准确性要求高
- ⚠ GPU显存需求较高 (~4-5GB)

5.6 应用场景

5.7 与其他一致性维度的关系

代码块

1

2

3

一致性维度层次结构


```
4
5 Overall Consistency (整体一致性)
6 |
7 |─ 内容一致性
8 |   |─ Subject Consistency (主体一致性)
9 |   |─ Background Consistency (背景一致性)
10 |
11 |─ 时序一致性
12 |   |─ Temporal Flickering (时序闪烁)
13 |   |─ Motion Smoothness (运动平滑度)
14 |
15 |─ 语义一致性
16 |   |─ Prompt Matching (与文本匹配)
```

5.8 实际应用示例

示例1: AI视频生成评测

代码块

```
1 from vbench import VBench
2
3 vbench = VBench('cuda', 'vbench/VBench_full_info.json', 'results')
4
5 # 评测生成视频与prompt的一致性# 假设有json文件包含prompt和视频路径的对应关系
6 vbench.evaluate(
7     videos_path='generated_videos/',
8     name='overall_consistency_test',
9     dimension_list=['overall_consistency'],
10    mode='custom_input'
11 )
```

示例2: 视频-文本对齐检查

代码块

```
1 import torch
2 from vbench.third_party.ViCLIP.viclip import ViCLIP
3
4 # 加载ViCLIP
5 viclip = ViCLIP(...).to('cuda')
6
7 # 准备数据
8 video_path = 'video.mp4'
9 prompt = 'A cat playing with a ball'# 计算一致性得分
```

```
10 frames = read_frames(video_path, num_frames=8)
11 video_feat = viclip.encode_vision(frames)
12 text_feat = viclip.encode_text(prompt)
13 score = (video_feat @ text_feat.T).item()
14
15 print(f'Consistency Score: {score:.4f}')
```

5.9 评分解读

注意: Overall Consistency的分数范围与其他维度不同，通常在0.1-0.4之间，这是由于视频-文本匹配的天然难度决定的。

6. 综合对比与选择指南

6.1 五个维度的对比表

6.2 评测维度选择决策树

```
代码块
1  开始评测
2      |
3      └─ 视频有文本描述?
4          └─ YES → Overall Consistency
5          |
6          └─ 关注纹理闪烁?
```

```
7      |   | — 静态场景 → Temporal Flickering
8      |   | — 运动场景 → 结合Flicker Penalty (worldscore)
9      |
10     | — 关注防抖性能?
11     |   | — YES → Motion Smoothness
12     |
13     | — 有明确主体对象?
14     |   | — YES → Subject Consistency
15     |
16     | — 关注背景稳定性?
17     |   | — YES → Background Consistency
```

6.3 场景推荐组合

场景1: AI视频生成评测

代码块

```
1  dimensions = [
2      'overall_consistency',      # 与prompt匹配度
      'subject_consistency',      # 主体稳定性
      'background_consistency',   # 背景稳定性
      'temporal_flickering'      # 闪烁检测
3  ]
```

场景2: 真实相机拍摄评测

代码块

```
1  dimensions = [
2      'temporal_flickering',      # 静态场景闪烁
      'motion_smoothness',        # 防抖性能
      'background_consistency'    # 背景稳定性
3  ]
```

场景3: 视频编辑质量评测

代码块

```
1  dimensions = [
2      'overall_consistency',      # 整体一致性
      'motion_smoothness',        # 运动平滑度
      'subject_consistency'      # 主体保持
3  ]
```

7. 技术总结

7.1 核心技术对比

7.2 计算复杂度分析

时间复杂度（处理1080p视频）：

代码块

1

Temporal Flickering:

$O(N)$

$N =$ 帧数

2

Motion Smoothness:

$O(N * M)$

$M =$ 模型推理时间

3

Subject Consistency:

$O(N * D)$

$D =$ DINO推理时间

4

Background Consistency:

$O(N * C)$

$C =$ CLIP推理时间

5

Overall Consistency:

$O(k * V)$

$k =$ 采样帧数(8), $V =$ ViCLIP推理时间

空间复杂度（GPU显存）：

代码块

1

Temporal Flickering:

~1GB

2

Motion Smoothness:

~5GB

3

Subject Consistency:

~3GB

4

Background Consistency:

~4GB

5

Overall Consistency:

~5GB

7.3 最佳实践建议

1. 评测前准备

- 确保GPU显存充足（建议8GB+）
- 下载所需的预训练模型
- 预处理视频（统一分辨率、帧率）

2. 评测策略

- 静态场景优先使用Temporal Flickering

- 运动场景必须使用Motion Smoothness或Flicker Penalty
- 根据评测目标选择合适的维度组合

3. 结果解读

- 综合多个维度的结果
- 注意不同维度的分数范围差异
- 结合具体应用场景判断

4. 性能优化

- 使用批量处理 (Background Consistency)
 - 启用特征缓存 (Overall Consistency)
 - 根据显存调整batch size
-

8. 参考资料

8.1 学术论文

1. VBench: Comprehensive Benchmark Suite for Video Generative Models

- VBench框架的核心论文
- <https://arxiv.org/abs/2311.17982>

2. Emerging Properties in Self-Supervised Vision Transformers (DINO)

- Subject Consistency使用的DINO模型
- <https://arxiv.org/abs/2104.14294>

3. Learning Transferable Visual Models From Natural Language Supervision (CLIP)

- Background Consistency使用的CLIP模型
- <https://arxiv.org/abs/2103.00020>

4. AMT: All-Pairs Multi-Field Transforms for Efficient Frame Interpolation

- Motion Smoothness使用的AMT模型
- <https://arxiv.org/abs/2304.09790>

8.2 代码仓库

- **VBench GitHub:** <https://github.com/Vchitect/VBench>
- **DINO GitHub:** <https://github.com/facebookresearch/dino>
- **CLIP GitHub:** <https://github.com/openai/CLIP>

- **AMT GitHub:** <https://github.com/MCG-NKU/AMT>

8.3 相关工具

- **RAFT (光流估计):** <https://github.com/princeton-vl/RAFT>
- **Flicker Penalty:** `vbench/worldscore/flicker_penalty.py`

9. 附录

9.1 完整评测示例代码

代码块

```
1  #!/usr/bin/env python3"""
2  VBench 完整评测示例
3  """from vbench import VBench
4
5  def comprehensive_evaluation(video_dir, output_dir='results'):
6      """
7      完整的VBench评测流程
8
9      Args:
10         video_dir: 视频目录
11         output_dir: 结果输出目录
12     """# 初始化VBench
13     vbench = VBench(
14         device='cuda',
15         full_info_dir='vbench/VBench_full_info.json',
16         output_path=output_dir
17     )
18
19     # 评测所有维度
20     dimensions = [
21         'temporal_flickering',
22         'motion_smoothness',
23         'subject_consistency',
24         'background_consistency',
25         'overall_consistency'
26     ]
27
28     results = {}
29
30     for dimension in dimensions:
31         print(f"\n评测维度: {dimension}")
32         print("=" * 60)
```

```

33
34         score, video_results = vbench.evaluate(
35             videos_path=video_dir,
36             name=f'{dimension}_eval',
37             dimension_list=[dimension],
38             mode='custom_input'
39         )
40
41         results[dimension] = {
42             'average_score': score,
43             'video_results': video_results
44         }
45
46         print(f"平均分数: {score:.4f}")
47
48     return results
49
50 if __name__ == '__main__':
51     results = comprehensive_evaluation('test_videos/')
52
53     # 打印汇总结果
54     print("\n" + "=" * 60)
55     print("评测结果汇总")
56     print("=" * 60)
57
58     for dimension, result in results.items():
59         print(f"{dimension:30s}: {result['average_score']:.4f}")

```

9.2 常见问题 FAQ

Q1: 为什么Temporal Flickering分数很低? A: 可能原因: 视频包含运动、相机抖动、或真实的内容变化。建议使用Motion Smoothness或Flicker Penalty代替。

Q2: Motion Smoothness评测太慢怎么办? A: 降低视频分辨率、减少帧数、或使用更强的GPU。

Q3: Overall Consistency需要什么样的文本prompt? A: 需要准确描述视频内容的文本。对于真实拍摄视频, 需要手动添加描述。

Q4: 不同维度的分数范围不同吗? A: 是的。Temporal Flickering等大多数维度范围是[0,1], 而Overall Consistency通常在[0.1,0.4]之间。

Q5: 如何选择合适的评测维度? A: 根据视频类型和评测目标选择。静态场景用Temporal Flickering, 运动场景用Motion Smoothness, 有prompt用Overall Consistency。

10. 更新日志

版本 v1.0 (2026-01-06)

- 初始版本发布
- 包含5个核心维度的详细说明
- 提供完整的技术文档和使用示例

文档编写: VBench技术文档团队

最后更新: 2026-01-06

文档版本: v1.0